

Session 7 : Broad et Narrow phases – Part 1

Objectifs de la session

- Comprendre le **pipeline de détection de collision** en trois étapes.
- Mesurer le budget temps d'une frame (16 ms) et pourquoi la physique doit tenir en 2 ms.
- Expliquer pourquoi la détection naïve est en $O(N^2)$ et pourquoi c'est inacceptable.
- Maîtriser l'AABB (Axis Aligned Bounding Box) comme fondement de la Broad Phase.
- Détailler trois structures de partitionnement : Grille Uniforme, Quadtree/Octree, Sweep-and-Prune.
- Comparer leurs complexités, avantages et cas d'usage.
- Introduire la Narrow Phase et le test de chevauchement AABB.

1. Le Pipeline de Détection de Collision

Trois étapes pour détecter et résoudre une collision

Un moteur physique moderne ne teste pas toutes les paires d'objets. Il suit un **pipeline en entonnoir** qui élimine progressivement les faux positifs :

1. **Broad Phase** — “Qui pourrait entrer en collision ?”
Réponse rapide avec des boîtes simplifiées (AABB). Élimine 95–99 % des paires.
2. **Narrow Phase** — “Ces deux objets se touchent-ils vraiment ?”
Test géométrique précis (sphère, OBB, GJK, etc.). Coûteux mais appliqué à très peu de paires.
3. **Collision Response** — “Que faire ?”
Impulsion, correction de position, friction (vu en Session 6).

💡 L'entonnoir en chiffres

Pour 1 000 objets, la brute force génère $\approx 500\,000$ paires. La Broad Phase en retient typiquement $\approx 2\,000$. La Narrow Phase confirme ≈ 200 collisions réelles. La Response n'en traite que les pertinentes. Chaque étage divise le travail par un ordre de grandeur.

2. Le Contexte : Le Budget Temps (16 ms)

Le budget temps d'une frame

À 60 FPS, on dispose de **16.6 millisecondes** pour tout calculer :

- Rendu : 8 ms
- IA / Gameplay : 4 ms
- Audio / Réseau : 2 ms
- **Physique : 2 ms**

Si la détection de collision prend 10 ms, le jeu saccade. L'optimisation est vitale.

💬 Les 2 ms de la physique

Sur ces 2 ms, la Broad Phase doit prendre **moins de 0.5 ms**. C'est elle qui scale avec N . La Narrow Phase et la Response consomment le reste. Si la Broad Phase est lente, tout le reste cascade — c'est le goulot d'étranglement numéro un des moteurs physiques.

3. Le Problème : La Malédiction $O(N^2)$

Détection naïve (brute force)

Avec N objets, chaque objet teste tous les autres. Le nombre de paires uniques est :

$$C = \frac{N(N-1)}{2} \approx O(N^2)$$

Objets (N)	Paires	Temps à 1 μ s/test
10	45	≈ 0.05 ms
100	4 950	≈ 5 ms
1 000	499 500	≈ 500 ms
10 000	49 995 000	≈ 50 s

Table 1: La croissance quadratique rend la brute force inutilisable au-delà de quelques dizaines d'objets.

Solution : réduire le nombre de paires **avant** les tests coûteux. C'est le rôle de la **Broad Phase**.

4. L'AABB : La Boîte Fondamentale

Axis Aligned Bounding Box (AABB)

Une AABB est le plus petit rectangle (2D) ou pavé (3D) **aligné sur les axes** qui contient un objet. Elle est définie par deux points :

$$\text{AABB} = \{\min(x_{\min}, y_{\min}, z_{\min}), \max(x_{\max}, y_{\max}, z_{\max})\}$$

💡 Pourquoi des boîtes alignées sur les axes ?

Parce que le test de chevauchement entre deux AABB est **trivial** — quelques comparaisons sur chaque axe, pas de produit vectoriel, pas de trigonométrie. C'est le test le plus rapide qui existe en 3D.

Test de chevauchement AABB vs AABB

Deux AABB se chevauchent si et seulement si leurs intervalles se chevauchent sur **tous** les axes :

$$\text{overlap}_x = (x_{\max}^A \geq x_{\min}^B) \wedge (x_{\max}^B \geq x_{\min}^A)$$

Si un seul axe ne se chevauche pas, il n'y a pas de collision. C'est le principe du **Separating Axis**.

Test AABB en JavaScript.

```
function aabbOverlap(a, b) {  
  // A et B sont des objets { min: Vector3, max: Vector3 }  
  if (a.max.x < b.min.x || b.max.x < a.min.x) return false;  
  if (a.max.y < b.min.y || b.max.y < a.min.y) return false;  
  if (a.max.z < b.min.z || b.max.z < a.min.z) return false;  
  return true; // Chevauchement sur les 3 axes  
}
```

Calcul d'une AABB pour une sphère : trivial — la boîte va de $c - r$ à $c + r$ sur chaque axe, où c est le centre et r le rayon.

Calcul d'une AABB pour un mesh complexe : on parcourt tous les sommets et on garde les min et max par axe. Ce calcul se fait **une fois** au chargement, puis on translate la boîte avec l'objet.

5. La Broad Phase : Structures de Partitionnement

Principe de la Broad Phase

On entoure chaque objet par son AABB, puis on utilise une **structure de partitionnement** pour trouver rapidement quelles paires d'AABB **pourraient** se chevaucher. On ne teste précisément que les candidates.

On présente ici les trois structures les plus utilisées en jeu vidéo.

5.A. Grille Uniforme (Spatial Hashing)

Principe

On découpe le monde en cases (cells) de taille fixe S . Chaque objet est rangé dans la case correspondant à sa position. Pour tester les collisions d'un objet, on ne regarde que les objets dans sa case et les cases **adjacentes** (8 en 2D, 26 en 3D).

Spatial Hashing — structure et requête.

```
class SpatialGrid {
  constructor(cellSize) {
    this.cellSize = cellSize;
    this.cells = new Map(); // clé "x,y,z" -> liste d'objets
  }

  getKey(pos) {
    const x = Math.floor(pos.x / this.cellSize);
    const y = Math.floor(pos.y / this.cellSize);
    const z = Math.floor(pos.z / this.cellSize);
    return `${x},${y},${z}`;
  }

  clear() { this.cells.clear(); }

  insert(obj) {
    const key = this.getKey(obj.position);
    if (!this.cells.has(key)) this.cells.set(key, []);
    this.cells.get(key).push(obj);
  }

  getNeighbors(pos) {
    const cx = Math.floor(pos.x / this.cellSize);
    const cy = Math.floor(pos.y / this.cellSize);
    const cz = Math.floor(pos.z / this.cellSize);
    const neighbors = [];
    for (let dx = -1; dx <= 1; dx++)
      for (let dy = -1; dy <= 1; dy++)
        for (let dz = -1; dz <= 1; dz++) {
          const cell = this.cells.get(`${cx+dx},${cy+dy},${cz+dz}`);
          if (cell) neighbors.push(...cell);
        }
  }
}
```

```

    }
    return neighbors;
}
}

```

💡 Choix de la taille de cellule

La taille idéale est $S \approx$ diamètre max des objets. Si S est trop petit, un objet chevauche plusieurs cases (overhead). Si S est trop grand, chaque case contient trop d'objets (le filtrage ne sert à rien).

- **Complexité** : $O(N)$ pour construire la grille, $O(K)$ par objet où K est le nombre d'objets dans les 27 cases voisines. Si la distribution est uniforme, K est constant $\rightarrow$ **total en $O(N)$** .
- **Avantage** : accès $O(1)$, très rapide, facile à coder, excellent pour des objets de taille similaire.
- **Inconvénient** : mauvais si les tailles varient beaucoup (un petit objet et un énorme bâtiment dans la même grille), ou si le monde est infini (il faut un dictionnaire au lieu d'un tableau).
- **Cas d'usage typique** : particules, billes, projectiles, jeux d'arène fermée.

5.B. Arbres Spatiaux (Quadtree / Octree)

Principe

On commence avec une grande boîte contenant tout le monde. Si elle contient plus de T objets (seuil), on la découpe en 4 (2D = Quadtree) ou 8 (3D = Octree) sous-boîtes égales. On recommence récursivement jusqu'à ce que chaque feuille contient au plus T objets.

Octree — insertion et requête.

```

class OctreeNode {
    constructor(min, max, depth = 0, maxDepth = 5, maxObjects = 8) {
        this.min = min; this.max = max;
        this.depth = depth;
        this.maxDepth = maxDepth;
        this.maxObjects = maxObjects;
        this.objects = [];
        this.children = null; // 8 sous-nœuds ou null
    }

    insert(obj) {
        if (this.children) {
            for (const child of this.children)
                if (child.contains(obj)) child.insert(obj);
            return;
        }
        this.objects.push(obj);
        if (this.objects.length > this.maxObjects
            && this.depth < this.maxDepth) {
            this.subdivide();
            // Redistribuer les objets existants
            for (const o of this.objects)
                for (const child of this.children)
                    if (child.contains(o)) child.insert(o);
            this.objects = [];
        }
    }
}

```

```

subdivide() {
  const mid = this.min.clone().add(this.max).multiplyScalar(0.5);
  // Créer 8 enfants (2^3 combinaisons de min/max par axe)
  this.children = [];
  for (let i = 0; i < 8; i++) {
    const childMin = new THREE.Vector3(
      (i & 1) ? mid.x : this.min.x,
      (i & 2) ? mid.y : this.min.y,
      (i & 4) ? mid.z : this.min.z
    );
    const childMax = new THREE.Vector3(
      (i & 1) ? this.max.x : mid.x,
      (i & 2) ? this.max.y : mid.y,
      (i & 4) ? this.max.z : mid.z
    );
    this.children.push(new OctreeNode(childMin, childMax, this.depth + 1,
      this.maxDepth, this.maxObjects));
  }
}

query(aabb, results = []) {
  if (!this.intersectsAABB(aabb)) return results;
  if (this.children) {
    for (const child of this.children) child.query(aabb, results);
  } else {
    results.push(...this.objects);
  }
  return results;
}
}

```

Reconstruction vs mise à jour

Dans un jeu où les objets bougent, deux stratégies existent :

- **Rebuild chaque frame** : on détruit et reconstruit l'arbre. Simple mais coûteux ($O(N \log N)$). Acceptable si N est modéré (< 10000).
- **Update incrémental** : on déplace les objets dans l'arbre. Plus complexe, mais nécessaire pour de très grands mondes.

La plupart des moteurs de jeu font un **rebuild** chaque frame : c'est plus simple et souvent plus rapide en pratique grâce au cache.

- **Complexité** : $O(N \log N)$ pour construire, $O(\log N + K)$ pour une requête.
- **Avantage** : s'adapte automatiquement à la densité — zones denses subdivisées davantage, zones vides non subdivisées.
- **Inconvénient** : coûteux à reconstruire/mettre à jour si les objets bougent beaucoup ; overhead mémoire pour les nœuds.
- **Cas d'usage typique** : mondes ouverts, objets de tailles variées, culling de frustum, raycasting.

5.C. Sweep and Prune (SAP)

Principe

On projette les débuts (min) et fins (max) de chaque AABB sur un axe (généralement X). On trie la liste des intervalles. En parcourant la liste triée, on maintient un ensemble “actif” d’intervalles qui n’ont pas encore été fermés. Deux intervalles actifs simultanément se chevauchent sur cet axe.

Sweep and Prune — algorithme 1D.

```
class SweepAndPrune {
  constructor() {
    this.intervals = []; // [{ id, value, isStart }]
  }

  update(objects) {
    this.intervals = [];
    for (const obj of objects) {
      this.intervals.push({ id: obj.id, value: obj.aabb.min.x, isStart: true });
      this.intervals.push({ id: obj.id, value: obj.aabb.max.x, isStart: false });
    }
    // Tri par position croissante
    this.intervals.sort((a, b) => a.value - b.value);
  }

  getPairs() {
    const pairs = [];
    const active = new Set();
    for (const interval of this.intervals) {
      if (interval.isStart) {
        // Cet objet démarre : il chevauche tous les actifs
        for (const id of active) {
          pairs.push([id, interval.id]);
          active.add(interval.id);
        }
      } else {
        active.delete(interval.id);
      }
    }
    return pairs;
  }
}
```

💡 Cohérence temporelle (temporal coherence)

D’une frame à l’autre, les objets bougent peu. La liste triée est donc **presque** la même. Au lieu d’un tri complet ($O(N \log N)$), on utilise un **tri par insertion** ($O(N)$ sur une liste presque triée). C’est ce qui rend SAP si rapide en pratique : le coût amorti est quasi linéaire.

📌 SAP multi-axe (SAP 3D)

La version 1D génère des **faux positifs** : deux objets peuvent se chevaucher sur X mais pas sur Y. La version 3D exécute SAP sur les trois axes et ne retient que les paires qui se chevauchent sur **tous** les axes. On peut aussi n’utiliser qu’un seul axe (le plus discriminant) et filtrer ensuite avec le test AABB complet.

- **Complexité** : $O(N \log N)$ au pire, $O(N)$ amorti grâce à la cohérence temporelle.
- **Avantage** : exploite la cohérence temporelle, pas de paramètre de taille de cellule à régler, excellent pour des objets de tailles variées.
- **Inconvénient** : dégénère si tous les objets sont alignés sur le même axe (la liste active devient énorme).
- **Cas d'usage typique** : moteurs physiques professionnels (Bullet, Box2D utilise une variante), objets de tailles hétérogènes.

6. Tableau Comparatif des Structures

Critère	Grille Uniforme	Quadtree/Octree	SAP
Complexité construction	$O(N)$	$O(N \log N)$	$O(N \log N)$ / $O(N)$ amorti
Complexité requête	$O(K)$ par objet	$O(\log N + K)$	$O(N + P)$ paires
Tailles hétérogènes	Mauvais	Bon	Bon
Monde infini	Possible (hash)	Difficile	Oui
Objets statiques	Excellent	Excellent	Bon
Objets dynamiques	Très bon	Coûteux (rebuild)	Excellent (cohérence)
Paramètres à régler	Taille de cellule	Seuil, profondeur max	Aucun
Difficulté d'implémentation	Facile	Moyenne	Moyenne

Table 2: Comparaison des trois structures de Broad Phase les plus courantes. K = objets voisins, P = paires candidates.

7. Introduction à la Narrow Phase

Le rôle de la Narrow Phase

La Broad Phase donne une liste de **paires candidates**. La Narrow Phase teste chaque paire avec une géométrie **précise** pour confirmer la collision et calculer la normale de contact.

Tests courants (du moins cher au plus cher) :

- **Sphère vs Sphère** : $\|c_A - c_B\| < r_A + r_B$. Un seul test, $O(1)$.
- **AABB vs AABB** : 6 comparaisons (vu section 4).
- **Sphère vs AABB** : distance du centre au boîtier, clampé sur chaque axe.
- **OBB vs OBB** (Oriented Bounding Box) : théorème des axes séparateurs (SAT), jusqu'à 15 axes en 3D.
- **Maillage vs Maillage** : GJK (Gilbert–Johnson–Keerthi) + EPA pour la pénétration. Coûteux, réservé aux paires confirmées.

💡 Hiérarchie de tests

On applique toujours les tests du **moins cher au plus cher**. Si le test AABB échoue, on s'arrête — inutile de lancer GJK. C'est la même philosophie que l'entonnoir : chaque étage élimine des candidats à moindre coût.

8. TP : Broad Phase — Spatial Hashing

💡 Objectif du TP

Implémenter une Broad Phase par **grille uniforme (Spatial Hashing)** et le test de chevauchement **AABB**, puis comparer les performances avec la brute force $O(N^2)$. Le fichier de départ `session07_Broad_Part_1.js` fournit la scène Three.js, la GUI et le moteur de collision — il ne reste qu'à compléter trois fonctions.

Mise en place

- Ouvrir `session07_Broad_Part_1.html` dans le navigateur (via Live Server).
- La scène contient un cube 3D rempli de billes en mouvement avec rebonds sur les parois.
- La GUI propose : nombre de billes (10–3000), toggle Broad Phase ON/OFF, restitution, vitesse du temps, reset.
- Un compteur **FPS** et **Tests/frame** est affiché en temps réel.
- Au départ, la Broad Phase est activée mais les fonctions sont vides — tout passe en brute force. C'est normal, c'est ce que vous allez corriger.

Missions

Mission 1 — `SpatialGrid.insert(obj)`

Ranger chaque objet dans la bonne cellule de la grille. Utiliser `getKey(obj.position)` pour obtenir la clé "x, y, z", créer le tableau si la cellule n'existe pas, puis ajouter l'objet.

Mission 2 — `SpatialGrid.getNeighbors(pos)`

Récupérer les objets situés dans la cellule de `pos` et les **26 cellules voisines** ($3 \times 3 \times 3$). Boucler sur `dx, dy, dz` de -1 à +1, construire la clé et récupérer le contenu. Retourner le tableau des voisins.

Mission 3 — `aabbOverlap(aabbA, aabbB)`

Implémenter le test de chevauchement AABB vu en section 4 : 6 comparaisons, une par axe. Renvoyer `false` dès qu'un axe ne se chevauche pas, `true` si les trois se chevauchent.

Mission 4 — Benchmark

Une fois les trois fonctions complétées, comparer Broad Phase ON vs OFF avec différents nombres de billes. Observer le compteur **Tests/frame** : il doit chuter drastiquement quand la Broad Phase est active.

Scénarios à observer

- **Faible densité (50 billes)** : la brute force reste fluide. La Broad Phase est légèrement plus rapide mais la différence est faible.
- **Forte densité (1000+ billes)** : la brute force s'effondre (≈ 500000 tests/frame), la Broad Phase reste fluide.

- **Extrême (3000 billes)** : avec Broad Phase ON, la simulation reste interactive ; avec OFF, elle devient un diaporama. C'est la démonstration concrète de l'entonnoir.
- **Compteur Tests/frame** : avec Broad Phase, le nombre de tests doit être proportionnel au nombre de **voisins réels**, pas à N^2 . Comparer les chiffres affichés.

⚠ Pièges à éviter

- Ne pas oublier d'appeler `grid.clear()` au début de chaque frame avant de réinsérer les objets.
- La fonction `getNeighbors` doit couvrir les 27 cellules ($3 \times 3 \times 3$), pas seulement la cellule courante — un objet à cheval entre deux cellules doit être détecté.
- Le test AABB doit renvoyer `false` **dès le premier axe qui ne se chevauche pas** — ne pas tester les trois axes si le premier échoue (optimisation).
- Faire attention aux **doublons** : le code fourni utilise `ballB.id <= ballA.id` pour éviter de tester chaque paire deux fois. Ne pas retirer cette garde.

Bonus (+2 pts)

Ajouter un **Octree** comme troisième option dans la GUI (en plus de Brute Force et Grille). Comparer les performances lorsque les billes sont concentrées dans un coin du cube (distribution non uniforme). L'Octree devrait mieux gérer cette situation car il subdivise les zones denses.